Day: Date:

'Leacture No 24'

Static	Dynamic
Static memory compile time per fix hoti hy.	Dynamic memory run time per allocate or manually free ki jati hy.

Dereference

Dereference ka matlab hy ky is ki memory ko delet kr dyna

Cast:-

ka matlab hy ky 1 datatype ko 2 dusry datatype my convirion.

1) Defination malloc()

malloc () is a function which is used to allocate 'a block of memory dynamically.'

2) Calloc()

Calloc() is a function which is used to allocate 'multiple blocks of memory.'

3) Realloc()

Realloc() is a function which is used to resize the memory block 'which is allocated by malloc or calloc before.'

Day: ate:

Memory leak

Memory leakage accours in C++ when programmers allocates memory by using new keyboard and forgets to deallocated the memory by using the delete() function or delete[] operator. One of the most memory leakage accours in C++ by using wrong delete operator.

Dangling pointers

Dangling pointers is a situation where you have valid pointers in the stack, but it is pointing to invalid memory. You might end up in this situation when you deallocate the cheap memory before the pointers in stack deallocated.

Day: Date:

allocate کرنا dynamic memory

same کام T/F کرتا calloc, malloc میں Papers

#include <iostream>

#include <conio.h>

malloc → #include <stdlib.h>

using namespace std;

main()

{

return 0;

}

handle کرنا dynamic memory

use کرنا *pointer کے

code

→ main()

{

int numbe_of_student, total_ages = 0, *ptr1, *ptr2;

cout<<"\n\n How many student ages you want to store?:";

Cin>> number_of_studen;

Ptr1 = (int *) malloc (number_of_st * sizeof(int));

if (Ptr1 == null)

{

cout<<"\n\n Dynamic memory not Allocated";

return 1;

}

Ptr2 = Ptr2;

getches();

return 0;

}

```
for (int i=1; i<= number_of_student; i++)
{
  cout << "\n\n Enter age of Student " << i << ": ";
  cin >> *ptr2;
  total_ages = total_ages + *ptr2;
  Ptr2++;
}
Cout << "\n\n Average Age: " << total_ages / num_of_s;
Free (Ptr1);
Ptr2 = Null;
return 0;
}
```

Day: Date:

Leacture no 25

→ Defalt function:-

```
#include <iostream>
#include <conio.h>
using namespace std;
void Test (int a=1, int b=2)   → age hum sab ki value dy to isy default argument kehty hy
{
cout << "value of a: " << a << " & value of b: " << b;
}
main()
{
Test ( );   → age ma yaha value Put krdo to isy Parameter kehty hy
getch ();
return 0;
}
```

Inline function:-

ایسے فنکشن جو کہ چھوٹے ہوتے ہیں function کو inline function

کہتے ہیں small peace of code

Day: ate: Day:

inline function:-

```
using namespace std;
inline void Max (int a, int b)
{
  if (a>b)
     cout<<"\n\n a is Greater ";
  else
     cout<<"\n\n b is Greater ";
}
main ()
{
    Max (10, 20);
    return 0;
}
```

Day: Date:

overloading functions:

- same written type کے ہوں multiple ایسے
- جن کے arguments کی تعداد same ہو لیکن data type مختلف ہو

```
using namespace std;
void Test()
{
    cout<<" Test function 1";
}
void Test (int a)
{
  cout<<"\nTest function 2 with 1 argument";
}
void Test( int a, int b)
{
  cout<<"\nTest function 3 with 2 argument";
}
main (){
Test ( );
Test (10);
Test (10, 20);
return 0;
}
```

Day: ate:

'Leacture no 26'

Data member:-

Data کو Data میں جن میں variables ہوتے ہیں Data member
جسے کہ Store کرتے ہیں۔

-> Variables
-> Arrays
-> Structure
-> Pointers

Functions:-

Function is a peace of code in which
Solve any problems. Solution.
problem ہوتی ہے body ایک ایسی Function
line of code کے۔ Solution کا
کہتے ہیں

Class:-

جب ہمارے function اور Data members ایک ہی جگہ پر آجائے
کلاس
ہو جاتے ہیں اسے ہم کہتے ہیں

Instances:-

جب بھی ہم کسی کلاس کو programming میں access کرتے ہیں
object کا variable بنانا پڑتا ہے اس کلاس کا
تو اس کلاس کا ایک
instance بنانا پڑتا ہے۔ اسے کہتے ہیں۔

Day: Date:

Definition of a class:

کلاس کسی body کے اندر پن بنتی کلاس ایک بہت بڑا پیکٹ ہے
کلاس بالکل آزاد بنتی ہے _ class ایک key word ہے کلاس
کا کوئی بھی نام رکھ سکتے ہیں لیکن
class Student
اس کے بعد کلاس کی body ہوتی ہے body کے بعد ; ہوتا ہے Semicolon

```
#include <iostream>
#include <conio.h>
using namespace std;
class Student
{
    // Data members (variable, pointer, Array, Structure)
    // Functions
};
main()
{
    return 0;
}
```

Voide dafult:

voide dafult pcaivate hota hy.
class ky under 3 private key word
hoty hy (1) Privale: (2) Protected:
(3) Public:

Type here to search

DELL

Day:

ate:

Private کے بنے اگر کوئی بھی چیز بناؤ وہ چیزیں صرف اسی کلاس کے اندر use ہو سکتی ہے یعنی جو student نام کی بنائی گے اس کلاس کی کی body سے باہر نہیں جا سکتے

Private:
bodef
mano
queen

Protected کے بنے جو بھی چیز بنائیں گے وہ اس کلاس کے اندر بھی use ہو گی اور اس کلاس کی جو child class بناتے ہیں اس میں بھی use کر سکتے ہیں child calls کے علاوہ اور کسی کلاس میں نہیں بنا سکتے

Public کے بنے جو چیز ہوتی ہے یہ چیزیں ہم ہر جگہ use کر سکتے ہیں ہم Pury project میں ہر جگہ use کر سکتے ہیں

⇐ اگر ہم نے Data member اور member function کے اوپر کوئی key word نہ لگے اگر ہم کچھ نہ لگے تو وہ by dafult private key/access ہے

class کی چیزوں کو access کرنے کے لیے سب سے پہلے اس کا object کہتے ہیں یعنی

S.SetData();
obj
func call

Day:

#include
#include
using
class
{
Priv
in
S
Pub
{
}
}

Day: Date:

Program

```
#include <iostream>
#include <conio.h>
using namespace std;
class student
{
private:
  int id;
  string name;
public:
  void setData()
  {
    id = 100;
    name = "Rida";
  }
  void GetData()
  {
    cout<<"Student id";<<id;
    cout<<"\n\n student name:"<<name;
  }
};
main() {
  student s;
  s.setData();
  s.getData();
  return 0;
}
```

Data Type

object

Day: ate:

کلاس کی body سے باہر سے define کرنے
اگر class کی body سے باہر آئے گا کہ کسی
function کی body کی آئی یعنی Define کرنا ہو بھی
return type کی آئی ہے پہلے سب سے
Name کی آئی ہو تو کیسے بنانے ہیں
ہو تو void کی return کی Voide کی
Scope resulation :: Then

Separation.

```
#include<iostream>
#include<conio.h>
using namespace std;
class student
{
  Privale:
    int id;
    string nam;
Public:
  void SetData();
  void GetData();
};
  void student::setData()
  {
  id = 100;
name = "Rida";
}
```

Day:

```
void Stude
{
  cou
  coa
}
main()
{
  S
  S.
  S.
  ge
}
C
C
int
Defualt
{
```

Day: Date:

```
void Student :: getData()
{
    cout << "Student id:" << id;
    cout << "\n\n Student name:" << name;
}
main()
{
    Student S;
    S.setData();
    S.getData();
    getch();
    return 0;
}
```

Constructors

Constructors ایک function ہے جو کلاس کے اندر ہوتی ہے۔ ان کی Data type بنانے نہیں value کو اگر initialize کرنا ہو کسی بھی variable کو value دینے کو initialize کرنا کہتے ہیں۔ یعنی **Defualt initialize** کرنا کہتے ہیں۔ Defualt کی Data mem کے کلاس کے۔ defualt initialize جو ہے int id = 0; کو constructor اور constructor کو کہتے ہیں۔ ~~value دینے کو~~ value set کرنے کو void, int کوئی بھی نہیں ہوتی یعنی return value نہیں ہوتی

Day: ate:

Constructor automatically call ho jata hy is ko call krny ki zaroorat nahi hy

Constructor call kab hota jab hmm call class ka object read krty hy

Constructors ki type:-

Default

Parameterized

Copy

وہ کنسٹرکٹر جو کوئی پیرامیٹر نہیں لیتا وہ ڈیفالٹ کنسٹرکٹر کہلاتا ہے

Constructor Ka program

```
using namespace std;
class Student
{
private:
  int id;
String name;
 Public:
  Student() ->        constructor
{ void setData();
   id = 0;
   name = " ";
}
```

Day: class key under

```
void setData();
void GetData();
};
void student::setData()
{
}
void student::getData()
{
  id = 100;
  name = "Rida"
}
void stud::getData()
{
  cout << "Stud id:" << id;
  cout << "\n\n stud name" << name;
}
main()
{
  Student s;
  s.SetData();
  s.getData();
  return 0;
}
```

class key Date: Datsidt

```
using namespace std;
class student
{
Private:
  int id;
  string name;
Public:
  Student();
  void SetData();
  void getData();
};
Student::Student()
{
  id = 0;
  name = "";
}
```

Day: ate:

'Leacture No 27'

constructor overloading:

ایک ہی کلاس میں simple constructor اور parameterized constructor دونوں بنا لینے کو constructor overloading کہتے ہیں۔

```
Public:
Student();          -> simple constructor
student (int i; string n);   -> parameterized
```

Destructors

Destructor ik function hy

Destructor ka name be same class wly name lki tara hota hy

Destructor by dafulte atomatically call hota hy

Destructor apna class ko distroy krty RAM sy

Destructor ki phanchan ~Student();

Day: Date:

Leacture No 29

Program 1

Friend function:-

```
#include<iostream>
#include<conio.h>
using namespace std;
Class A
{
  Privat:
    int number;
  Public:
Structor A()
  {
    number = 0;
  }
  friend void addnumber(A);
};
void addnumber(A obj)
{
  obj.number += 5;
  Cout << obj.number;
}
main()
{
  A obj;
addnumber(obj);
getch();
return 0;
}
```

Day:

Friend Class

```
#include<iostream>
#include<conio.h>
using namespace std;
class B;
class A
{
  Private:
  int number;
  Public:
  A()
  {
  number=10;
  }
  Friend class B;
};
class B
{
  Private:
  int number;
  A obj;
  Public:
  B()
  {
  number=15;
  }
  void add number()
  {
  Cout<<obj.num+numb;
  }
};
main()
{
  B obj;
  obj.addnumber();
  return 0;
}
```

Day: Date:

'Leacture NO 30'

Reference variable

```
main()
{
int i = 100;
int &j = i;
j = 129;
cout << i;
```

ایک variable کے لیے دو Box

Pointer

```
main ()
{
int i = 100;
int *ptr = &i;
cout << Ptr; // Address of variable
cout << "\n" << *ptr; // value of variable i
cout << "\n\n" << &Ptr; // Address of pointer
```

ایک variable کے لیے ایک Box

Day:

Leacture no 31

Mathamatically overloading :- programm

if void function کو return کرنا ہو تب ہم نے کیا ہو

if Data type (float, int) کو return کرنا ہو تو ہم نے کیا ہو

```
# include <iostream>
# include <conio.h>
using namespace std;
class A
{
  Private:
    int number;
  Public:
  A()
  {
    number = 5;
  }
  void operator ++()
  {
    ++number;
    cout << "number:" << number;
  }
};
main() {
  A obj;
  ++obj;
  return 0;
}
```

operator کو implement کیا۔ screen پر display کیا۔

Day:

No

```
#inc
#inc
#inc
using
Cla
{
};
main
{
cal
++o
ret
}
```

call →

Day: Date:

Non-Mathamatically overloading

Non-mathamatically وہ مطلب ہے جب آپ نے اپنی کسی بھی string کے اوپر overloading کی ہو

```
#include<iostream>
#include<conio.h>
#include<string.h>
using namespace std;
class addString
{
  Private:
    char Str1[25];
    char Str2[25];
  Public:
    addString()
    {
      Strcopy(Str1, "Rida");
      Strcopy(Str2, "Alia");
    }
    void operator +()
    {
      cout<<"new String :"<<Strcat(Str1, Str2);
    }
};
main()
{
  addString obj;
  +obj;
  return 0;
}
```

Array کی ایک Type جو کہ Character ہے اور جس میں ہم Store کرتے ہیں Char کو

String Char کی طرح کسی بھی Store کو value کرنا ہو تو اس کے لیے ہمارے پاس built in fun ہوتا ہے

Strcopy کے اندر ہم نے 2 چیزیں pass کرنی ہوتی ہیں پہلا Store کرنے کی value جس میں ہم Array کی value store کرتے ہیں اور دوسرا Str1 اور Str2 کے اندر جو value ہے یعنی Rida

overload

call

call

Day: ate:

'Leacture No 32'

Substruct operator overloading :-

```
#include <iostream>
#include <conio.h>
using namespace std;
class A
{
  Private:
   int number;
  Public:
   A(int a)
   {
      number = a;
   }
   void operator -(A obj)
   {
   cout<<"Answer= "<<number-obj.number;
   }
};
main()
{
A obj1(10), obj2(5);
obj1 - obj2;
```

obj2 is argument jo will taka by

obj 1 is function jo call ker taha hy

Day: Date:

unary operators:-

```
#include <iostream>
using namespace std;
class A
{
  private :
   int number;
  public:
   A()
  {
    number = 5;
  }
   void operator ++()
  {
    ++number; // add 1
    cout<<"number = "<<number;
  }
};
main()
{
 A obj
 ++obj;
 return 0;
}
```

Day: ate:

'Leacture No 33'

Assignment operator :-

```
#include<iostream>
#include<conio.h>
using namespace std;
Class A
{
Private:
  String name;
Public:
  A(String n)
  {
    name = n;
  }
  void display()
  {
    Cout<<name;
  }
};
main()
{
A obj1("Rida"), obj2("warda");
Cout<<"obj1 Name:";
obj1.display();
Cout<<"Obj2 Name:";
Cout Obj2.display();
return 0;
}
```

```
Cout<<"\n\n after Assign name obj1 to obj2";
obj2 = obj1;
obj2.display();
```

very simple program my neater very

Day:

Assignment o

```
using namespace
Class A
{
  Private:
    String N
  Public:
  A(String n),
  {
    name = n;
  }
  A
  void ope
  {
    return *
    obj1
    name =
  }
  void disp
  {
    Cout
  }
};
main()
{
```

overloading operator,

Day: Date:

Assignment operator overloading :

```
using namespace std;
class A
{
Private:
    String Name;
Public:
A (String n)
{
  name = n;
}
void operator = (A obj)
{
  name = obj.name;
}
void display()
{
  cout << name;
}
};
main()
{
  A obj1("Rida"), obj2("Alia");
  cout << "obj1 Name:";
  obj1.display(); // member function
  cout << "\n\n obj2 Name:";
  obj2.display();
  cout << "\n\n After Assign name obj2 to obj2:";
  obj2 = obj1;
  obj2.display();
  return 0;
}
```

جب ہم اپنے کلاس کا اور اپنے Data member کا نام same رکھ دیتے ہیں تو ہم اپنے Data کے لیے argument / member کے لیے differentiation کرنے کے لیے This کا keyword use کرتے ہیں۔ This -> This کا مطلب A کی طرف

Example

```
Public:
A (String name)
{
  This-> name = name;
}
```

Date:

Day:

Leacture No 34

Arrays of object

جب ہم اپنی کلاس کے object کی ایک سے زیادہ objects بنانے ہو مثال کے طور پر میں چاہوں ایک object اور اسکے تین objects بنانے ہیں

A obj1, obj2, obj3;

اگر objects کو بنانے ہو تو array کے object کے لیے

A obj[50]; // obj[0] obj[1] --- obj[49];

overloading of new and delete operators --> کی Example:

#include<ioStream>

#include<conio.h>
#include<StdLib.h> → جس کی header file میں جس کی under having free function کی code پڑا ہوتا ہے
using namespace Std;

Class A
{

Private:

int number;

Public:

A(int numeber) → جو argument کی data member کی value ہے This کی body ہو same

{

This -> number = number;

}

new op over load

Delet op ko over load

Day: Date:

obj کے لئے جب بھی new op کو overload کرنا ہو تو ہم بتاتے ہیں کہ کتنے obj کے لئے
تو زبان پر ہم اپنا keyword use کرتے ہیں۔ use کرتے ہیں

new op overload

```
void* operator new(size_t size)
{
    void *Ptr = ::operator new(size);
    return Ptr;
}
```

جب بھی ہم new اور delete کو overload کرنا ہو تو void* use کرتے ہیں۔
Address جو new op واپس کرتا ہے return کرتا ہے۔

delet op no overload

```
void operator delete(void * Ptr)
{
    Free(Ptr);
}

void display()
{
    cout << "number = " << Number;
};

main()
{
    A *Ptr;
    Ptr = New A(20);
    Ptr -> display();
    delete ptr;
    return 0;
}
```

Day:

Leacture No 35

Stream

Input take کرنے اور output display کرنے کے لیے C++ language میں جو objects استعمال کیے جاتے ہیں انہیں Stream objects کہتے ہیں۔

Source	Destination
Input	output

Input/output Stream ki Example

Single میں getch() یا get() کو character کو get کرتا ہے

```
#include <iostream>
#include <conio.h>
using namespace std;
main()
{
  int a;
  cout << "Enter Your value.";
  cin >>a;
  cout << "\n\n Your value is: " << a;
  getch();
  return 0;
}
```

Day:

Method

Cin.get()

Cin.getline

Cin.read()

Cout.wri

Cin-get

```
main()
{
  Char
  cout << "En
  Cin.get(
  cout << "
  get ch
  return
}
```

Day: Date:

Method of Stream:-

Cin.get() — use ہوتا ہے Stream میں تب کو Cin.get() ① کرتے ہیں جب ہمیں Single character input کو take کرنا ہو۔

Cin.getline() — use کرتے ہیں تب کو Cin.getline() ② Multiple words with Space جب get کرنے ہو یعنی get کرنی ہو paragraph کو get کرنا ہو۔

Cin.read() — use کرتے ہیں جب ③ اس کو ہم تب read کرنا ہو input کو user سے

cout.write() — use کرتے ہیں ④ اس کو ہم تب جب ہم نے اپنی output کو Screen پر display کرنی ہو۔

Cin.get()

```
main()
{
  char ch;
cout<<"Enter Single char:";
cin.get(ch);
cout<<"\n\n"<<ch;
getch();
return 0;
}
```

cin.getline()

```
char buffer[20];
cout<<"Enter your name:";
cin.getline(buffer, 20);
cout<<"\nYour name:"<<buffer;

getch();
return 0;
}
```

Day: ate:

Cin.read()

```
main()
{
Char buffer[13];
Cout << "Enter your name: ";
Cin.read(buffer, 13);
Cout << "\n\n your name: " << buffer;
getch();
return 0;
}
```

Cin.write()

```
main()
{
char buffer[13];
Cout << "Enter your name: ";
Cin.write(buffer, 13);
Cout << "\n\n your name: " << buffer;
getch();
return 0;
}
```

Day:

Stream

output

iostream

fstream

iomanip

Day: _____ Date: _____

"Lecture No 36"

Stream Mainpulation:-

output or Input جب ہم کلاس کا Stream mainpulation
کرتے ہیں تو اس کے لئے different C++ میں use
کرتے ہیں Stream Input/output کو use کرنے کے لئے header files
کو mainpulate کرنے کے لئے اس کے لئے سب سے پہلے ہمیں
header file کو

iostream.h → Simple Input/oputp kry use ki jati hy

fstream.h → is ko hmm tab use krty hy jab hmm file heandling krni ho Yani opna jo record hy I/O ko apni file ky under talchni ho Yeh file sy get krni ho.

iomanip.h → Yeh ik asi header file hy jo I/O Ki mainpulation krti hy mainpulation ky matlab hy (کسی چیز کو خوبصورتی سے دکھانا) I will define tariky sy apni cheez ko arrange krna apni I/O ko.

Day: ate:

cin.eof(); ⟶ it will return the state of end of file.

Mainpulators:

Mainpulators ایک ایسے key words ہوتے ہیں جو کہ آپ کی programming کے اندر اپنے variable یا operations کے behaviour کو change کرنے کے لیے use کرتے ہیں۔

Two Types of mainpulators:

(1) Non-Parameterized Mainpulators.
(2) Parametrized mainpulators

Non-para-zed maip-tors

Some of the manipulators are used without the argument

Base کو Decimal میں convert کرتے ہیں جب ہمیں Non-parameterized manipulators سے octal میں جانا ہو تو use کرتے ہیں۔

Day: Non-Pa

Examp

#include

#include

#includ

using

main ()

{

int

cau

get

set

}

Paraame

Some

with

ان کے ساتھ

Day: Non-Parametrized Date:

Example:-

```
#include <iostream>
#include <iomanip>
#includ <Conio.h>
using namespace std;
main ()
{
    int number = 12;
    cout << oct << number;
    getch();
    return 0;
}
```

decimal

Parametrized manipulator:-

Some of the manipulators are used with the argument

ایسے manipulator جو working کرنے سے پہلے argument کو pass کیا جاتا ہے

Day: ate:

Day:

Parametrized of Example:

```
#include <iostream>
#include <iomanip>
#include <conio.h>
using namespace std;
main()
{
    cout << setw(10) << "Rida Rasool";
    getch();
    return 0;
}
```

Day: Date:

"Lecture NO 37"

overloading insertion and Extraction op

Insertion	Extraction
Insertion means Less than operator output stream	Extraction means Geo Greterthan operator Input Stream ky liy use hota hy

جب بھی ہم نے use کرنے ہیں insertion یا Extraction operator کو overload کرنے ہیں تو اس کے لیے ہم friend function کا use کرتے ہیں۔

1. پہلے ہم Friend Type کرتے ہیں۔
2. Stream کی return type بناتے ہیں return Type کی function والی use کرتے ہیں۔

in Friend istream& operator >> (istream& in, Data& obj); (object name)

use کو Friend function کرنے کے لیے operator کا istream لیے کرتے ہیں۔

3. class کو operators ان دونوں Extraction اور insertion سے باہر جا کے define کرنا ہے

④ جب ان کو class سے باہر define کرتے ہیں اور body بناتے ہیں تو ان کے آگے ان نہیں لگاتے ہیں۔

⑤ اور friend fun کو class کے اندر لکھتے ہیں class سے باہر نہیں۔

⑥ اور اس میں ہم نے cinss نہیں لکھنا بلکہ جو ہم نے ostream / istream کا نوٹ بنایا ہے (in) نام سے وہ لکھنا ہے cin کی جگہ پر۔

Day: ate:

Friend function کی مدد سے Overloading کرنا

Extraction op

```
#include<iostream>
#include<conio.h>
using namespace std;
class myClass
{
  Private:
    int id;
    String name;
  Public;
  myClass()
  {
    id= 0;
    name = " ";
  }
// Extraction operator overload
Friend istream& operator >>(istream &in,
  myClass &obj);
};
```

insertion op

```
#include<iostream>
#include<conio.h>
using namespace std;
class myClass.
{
  Private;
    int id;
    String name;
  Public;
    myClass()
  {
    id = 0;
    name = " ";
  }
// Insertion operator overload
Friend ostream operator<<(ostream&out,
  myClass & obj);
};
```

Day:

next over

```
istream&op
myClass
{
cout<<"E
in>> obj
cout<<"
in>>ob
retur
}
main
{
my
Cin
ge
re
}
```

next overload:

Day: Date:

```
istream& operator>> (istream &in,
myClass &obj)
{
cout<<"Enter your ID:";
in>> obj.id;
cout<<"Enter your name:";
in>>obj.name;
return in;
}
main()
{
myclass obj;
cin>>obj;
getch();
return 0;
}
```

```
ostream& operator<< (ostream &out, myClass
& obj)
{
cout<<"Enter your name:"<<obj.name;
out<<"\n\n Enter your Id:"<<obj.Id;
retur out;
}
main()
{
myClass obj;
cout<<obj;
getch();
return 0;
}
```

Day:

"Lecture no 38"

Define of user Defined Manipulators:-

Predefined Manipulators are created by the user.

Example:

```
ostream& manipulator_name (ostream& os)
```

→ Programming by user Defined Manipulators

Predefined manipulators ↓

```
#include<iostream>
#include<conio.h>
using namespace std;
main()
{
 cout<<"\n Rida Rasool";
 getch();
 return 0;
}
```

مثلاً setw, setf, \n جیسے کوئی function predefined ہے مگر user defined fun بنا سکتے ہیں

Day:

user de

#includeiostrea

#include<conio.

using namesp

ostream& newl

{

out<<"\n

return

}

main()

{

cout<<

getch

return

}

Day: Date:

user defined Manipulator function

① جب کوئی user defined manipulator function بنانا ہو تو اس کا return type ostream کے reference operator کے ساتھ ہونا ضروری ہے۔ user defined manipulator function کی argument list میں ostream کا reference ہونا ضروری ہے۔

Example:

```
ostream& newline(ostream &out)
```

```
#include<iostream>
#include<conio.h>
using namespace std;
ostream& newline(ostream &out)
{
  out << "\n";
  return out;
}
main()
{
  cout << newline << newline << "Rida Rahool";
  getch();
  return 0;
}
```

"\t" (Tab) کے علاوہ بھی کئی اور escape sequences ہوتے ہیں۔

"\b" Back space

"\a" (Beep) (Alarm)

Day: ate:

Static keyword:

When Static keyword is used. variable or data members of functions can not be modified again. It is allocated for the lifetime of program. Static functions can be called directly by using class name. Static variables are initialized only once.

Example:

```
Static int i;
```

Day: _____ Date: _____

Static

```
#include<iostream>
#include<conio.h>
using namespace std;
void StaticFun()
{
  Static int i;
    i++;
  cout<<i<<"\t";
}
main()
{
cout<<"calling static function:";
StaticFun();
StaticFun();
getch();
return 0;
}
```

nonStatic

```
#include<iostream>
#include<conio.h>
using namespace std;
void nonStaticFun()
{
  int i;
  i++;
  cout<<i<<" ";
}
main()
{
cout<<"calling non static function:";
nonstatic Fun();
nonstatic Fun();
getch();
return 0;
}
```

اگر ہم نے کسی function کو دو دفعہ call کرنا ہو
تو اس کے لیے کونسی شرط ہے کہ وہ
ہو

Day: ate:

"Lecture no 39"

Pointer:-

Special ایک Pointer وہ variable ہوتا ہے جس کے اندر کسی بھی value کو store نہیں کرتے بلکہ ہم کسی نہ کسی memory location کا address store کرتے ہیں۔

Code of pointer

```
#include<iostream>
#include<conio.h>
using namespace std;
main()
    {
      int a = 10;
      int *ptr;
      Ptr = &a;
      cout<<ptr;
      getch();
      return 0;
}
```

Day: Date:

Reference

کسی address یا memory location کو refer کرنا Reference

Pointer کی طرح location

Call by value

when pass value as a parameters to the function.

جب بھی کسی function کو call کرتے ہیں تو اس کو parameter as a value دیتے ہیں

- call by ~~Reference~~ value

function of call by value:

```
#include<iostream>
using namespace std;
Void fun(int a, int b)
{
    cout<< a +b;
}
main()
{
  fun(10, 20);    // call by value
  return 0;
}
```

Day: ate:

Call by refrance

when pass refrance of the variable & object as a parameters to the function.

function کو call کرتے وقت جب ہم کسی variable کا refrance یا کسی object کے class کو کسی بھی object یا variable کو pass کرتے ہیں تو اسے call by refrence کہتے ہیں pass

Call by Refrence

```
#include<iostream>
using namespace std;
void Fun (int &a, int &b)
{
    cout<<a+b;
}
main()
{
  int a= 10 , b = 20;
  Fun(a, b);      // call by Refrence
  return 0;
}
```

Day: Date:

Dynamic Memory Allocation:

Allocate the memory using new keyword
Release the memory using delete keyword

جب ہم C++ کے اندر اپنی memory allocate کرتے ہیں
Dynamic memory Allocate ہم کرتے ہیں using new keyword
کے ساتھ

اور جب ہم delete keyword کو use کرتے ہیں

Allocation

```
#include<iostream>
using namespace std;
main()
{
   int *ptr;
  Ptr = new int; //Dynamic M/A
  return 0;
}
```

DeAllocation

```
#include<iostream>
using namespace std;
main()
{
int *ptr;
ptr = new int;
delete ptr; // DeAllocation
return 0;
}
```

Day: ate:

Assignment and Initialization

```
#include <iostream>
using namespace std;
main()
{
int a;   // Declare a value
int b = 20;  // Declaration with initialization
int c;
c = 20;   // Assign the value
return 0;
}
```

Copy Constructor:-

جب ہم C++ میں ایک object بناتے ہیں اور اس کی بہت سی copy دوسرے object کی data کو copy کرنا چاہتے ہیں تب ہم constructor کا استعمال کرتے ہیں۔ جیسے مثال کے طور پر میرے پاس ایک notebook ہے اس کی میں نے بہت سی copy کروانے ہیں b1 لیے تو وہ ہے جس سے ہم نے copy کروانی b2

Day: Date:

Copy Constructor

```
#include<iostream>
using namespace std;
Class myClass
{
    Private,
      in a;
    Public:
    myClass(int num)
    {
        a = num;
    }
    void display()
    {
    Cout<< a << "ft";
    }
};
main(){
 myClass obj1(38);
 myClass obj2 = obj1;
 obj1.display();
 obj2.display();
 return 0; }
```

Day: ate:

Rules for using Dynamic memory Allocation:

جب ہم کلاس کے اندر جا کے Dynamic memory کو Allocation کرتے ہیں، اس کے کچھ rules ہیں،

(1) jab hmm class banaty hy to us ky under constructor must banana hy

(2) second rule, Ap ko apni class ky under value assign krny ky liy assignment operator ko declear krna parta hy

(3) jab hmm apni class ky under dynamic memory allocation krty hy to must hy ky ap destructor be create kro

Day: Date:

"Lecture no 40"

object as class member

اس کا مطلب ہے ایک کلاس کے object کو دوسری کلاس کے obj میں پاس کرنا ہے create کرنا ہے۔

```
#include <iostream>
using namespace std;
class A
{
  public:
    A()
    { }
    void display()
    {
      cout<<"class A Function";
    }
    ~A()
    { }
};
class B
{
  private:
    A obj;
    B()
    { }
    void display()
    {
      obj.display()
      cout<<"\nIn Class B Function";
    }
    ~B()
    { }
};
main()
{
  B obj;
  obj.display();
  getch();
  return 0;
}
```

destructor

Day: Date:

Structures as class members

```
#include <iostream>
using namespace std;
struct student
{
    int id;
    string name;
};
class myClass
{
  private:
    student s;
  public:
    myClass()
    {
      s.id = 100;
      s.nam = "Rida";
    }
    void display()
    {
      cout << "student ID: " << s.id;
      cout << "\n\n student name: " << s.nam;
    }
    ~myClass()
    {}
};
main()
{
  myClass obj;
  obj.display();
  getch();
  return 0;
}
```

Classes inside classes

```
#include<iostream>
using namespace std;
class mainClass
{
 Public:
  mainClass()
  {}
 class subclass
   Public:
   subClass()
   {}
   void display()
   {
    cout<<"sub class Function";
   }
   ~subClass()
   { }
};
 void display()
 {
  cout<<"main class Function";
 }
~mainClass()
 { }
};
main()
{
 mainClass obj;
 mainClass::subclass obj1;
 obj.display();
 obj1.display();
 return 0;
}
```

scope resolution OP

Day: Date:

Lecture No 41

Template Functions:

Template is a generic data type.

Two Types of template function:

1. Function templates.
2. Class templates.

data types کا مطلب ہے کہ یہ ہر طرح کی Generic data type کو accept کر سکتا ہے بس بار بار data type change کرنے کی ضرورت نہیں پڑتی۔

```
#include<iostream>
using namespace std;
template< class T>
void Test (T a)
{
    Cout<<a;
}
main ()
{
  Test (33.0);
  getch ();
  return 0;
}
```

Day:

Overloading template Function

جب آپ ایک سے زیادہ Template Function بناتے ہیں
تو ان کا نام ایک رکھتے ہیں Template Function اور
same رکھتے ہیں return type اور Templat
argument کی تعداد same رکھتے ہیں تو ایسی صورت میں Templat Fun
کو overloading کہتے ہیں num of arg میں ان کی Templat Fun
data Type مختلف ہو یا
Template Fun کہتے ہیں

```
#include<iostream>
using namespace std;
template<class T>
Void Test(T a)
{
   Cout<<a;
}
void Test (T a, T b)
{
   cout<<"\n\n"<<a<<" "<<b;
}
Void Test (T a, T b, T c)
{
  Cout<<"\n\n"<<a<<" "<<b<<" "<<c;
}
```

Day: Date:

```
main ()
{
 Test (33);
Test (23, 77, 88);
 Test (23, 77);
 return 0;
}
```

Explicitly Specifying the type in a template function:

Template function یا کسی function کو value دی جائے گی اسی type کی ہوگی جس type کی pass کرنے میں یعنی جس type کی دی جائے گی۔ اور جو return value کرنے میں وہ بھی وہی ہوگی۔

```
#include <iostream>
#include<conio.h>
using namespace std;
template < class T, class S>
T test (S var)
{
 return "Hello" + Var;
```

T Test (S)
جیسا as a returning T
function ہے Then وہ
as a کو S اور پھر
وہی assignment

Day: ate:

```
using namespace std;
template <class T, class S>
T Test (S var)
{
 cout <<var;
 return "\n\n Hello Rida ";    //value 20
}
main()
{
cout<< Test<string , int >(20);
getch();
return 0;
}
```

Day: _____ Date: _____

Template Function and object

جب بھی ہم اپنا کوئی Template Function کی کلاس
Templat Function and object کو ہم آگے
کہاں بنا سکتے ہیں تو ہم آگے
بتائے ہیں۔

```
#include<iostream>
using namespace std;
Teplate <class T>
class myClass.
{
    Public:
    myClass()
    {}
    void display(T msg)
    {
        Cout << msg;
    }
    ~myClass()
    {}
};
main() {
        myClass<string> obj;
    cout<< obj. display("Hell Rida");
        return 0;
    }
```

"Lecture No 42"

Class Templates:-

```
#include<iostream>
using namespace std;
Template< class T>
class myClass
{
    T a;
    Public:
    myClass(T value)
    {
      a = value;
    }
  void display()
  {
   cout<<a;
  }
~myClass()
 {}
};
```

```
main()
{
 myClass<int>obj(39);
obj.display();
getch();
return 0;
}
```

Day: Date:

Templates and Static Members:

جب ہم data member کو static بناتے ہیں تو اس کی value کو خود سے initialize کرتے ہیں اور اس کو class سے باہر جا کے initialize کرتے ہے

```
#include<iostream>
using namespace std;
template <class T>
class myClass
{
    static T a;
    Public:
    myClass()
    {}
    void display()
    {
    cout<<a;
    }
    ~myClass
    {}
};
Template<class T>
T myClass<T> :: a = 35;
```

```
main()
{
myClass<int> obj;
obj.display();
return 0;
}
```

Day: ate:

Templates and friend Function:-

```
#include <iostream>
using namespace std;
template <class T>
class myClass()
{
    T a;
  Public:
    myClass()
    {
      a=100;
    }
    Friend void display();
    ~myClass()
    {}
};

void display()
{
  myClass<int> obj;
  Cout << obj.a;
}
```

```
main()
{
  display();
  return 0;
}
```

Day:

Ment
hmng
Parameterized
constructor
bang
ng

Day: Date:

"Lecture 44"

Matrix Class

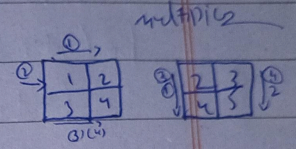
```
#include <iostream>
using namespace std;
Class Matrix
{
  Private:
    int A[2][2] , B[2],[2]
  Public:
    Matrix()
    {
      for(int i=0 ; i<=1; i++)
        for(int j=0; j<=1; j++)
        {
          A[i][j] = 0;
          B[i][j] = 0;
        }
    }
    Matrix(int m[2][2] , int n[2][2])
    {
      for(int i=0 ; i<=1 ; i++)
      for (int j=0; j<=1; j++)
```

Next hmny Parameterized constructor bany ny

```
        {
        A[i][j] = m[i][j];
        B[i][j] = n[i][j];
        }
    }
    void SumMatrix()
    {
      for (int i=0 ; i<=1 ; i++)
      {
        for(int j=0 ; j<=1; j++)
        {
          cout << A[i][j] + B[i][j] << " ";
        }
        cout <<"\n";
    } }
    void SubMatrix()
    {
      for(int i=0; i<=1; i++)
      {
        for(int j =0; j<=1; j++)
        {
          cout << A[i][j] - B[i][j] << " ";
        }
```

Day: Date:

```
        cout<<"\n";
    }
}
void mulMatrix()
{
    for(int i=0; i<=1; i++)
    {
        for(int j=0; j<=1; j++)
        {
            cout<<(A[i][0]*B[0][j])+(A[i][1]*B[1][j])<<" ";
        }
        cout<<"\n";
    }
}
void divMatrix()
{
    int temp[2][2], detB;
    for(int i=0; i<=1; i++)
        for(int j=0; j<=1; j++)
            temp[i][j] = B[i][j];
detB = (temp[0][0]*temp[1][1]) - (temp[0][1]*temp[1][0]);
```



```
temp[0][0] = temp[0][0] + temp[1][1];
temp[1][1] = temp[0][0] - temp[1][1];
temp[0][0] = temp[0][0] - temp[1][1];
temp[0][1] = temp[0][1] * (-1);
temp[1][0] = temp[1][0] * (-1);
 for(int i=0; i<=1; i++)
    for(int j=0; j<=1; j++)
       temp[i][j] *= detB;
 for(int i=0; i<=1; i++)
 {
cout<<(A[i][0] * temp[0][1]) + (A[i][1]*temp[1][j])<<" ";
 }
 cout<<"\n";
}
}
void transposeA()
{
 for(int i=0; i<=1; i++)
{
   for(int j=0; j<=1; j++)
   {
   cout<<B[j][i]<<" ";
   }
```

Day: Date:

```
    cout<<"\n";
    }
  }
  ~Matrix()
   {}
};
 main()
{
  int a[2][2]={{3,3},{1,1}}, b[2][2]={{2,1},{0,1}};
  Matrix obj(a,b);
  cout<<"2 by 2 Matrix addition:\n\n";
  obj.sumMatrix();
  cout<<"\n\n2 by 2 Matrix subtraction:\n\n";
  obj.subMatrix();
  cout<<"\n\n 2 by Matrix multiplication:\n\n";
  obj.mulMatrix();
 cout<<"\n\n 2 by 2 Matrix Division:\n\n";
 obj.divMatrix();
 cout<<"\n\n 2 by 2 Matrix A transpose:\n\n";
 obj.transposeA();
 cout<<"\n\n 2 by 2 Matrix B transpose:\n\n";
 obj.transposeB();
 return 0;
}
```